A Nominal Formalization of the Confluence of a Calculus with Explicit Substitutions via the Z Property

Flávio L. C. de Moura[1 [0000−0002−9390−5751]]

Departamento de Ciência da Computação
Universidade de Brasília, Brasília, Brazil
flaviomoura@unb.br

Abstract. The Z property is a compact proof technique for confluence that has been used to give elegant confluence proofs for the λ-calculus and several of its extensions. Previous formalizations of the (compositional) Z property, as well as of the Substitution Lemma for calculi with explicit substitutions, were developed in Rocq by the author. In this work, we formalize the λ_x-calculus, an extension of the λ-calculus with explicit substitutions, and then we apply the formalization of the compositional Z property to give a fully mechanized proof of confluence for the λ_x-calculus. To the best of our knowledge, this is the first formalization using the compositional Z property. The formalization is done in the Rocq prover (version 9.1.1), and is freely available (zenodo???). It comprises around 3 500 lines of Rocq code and 150 lemmas and theorems, and provides a reusable nominal toolkit for confluence proofs of calculi with explicit substitutions.

Keywords: Nominal syntax, Confluence, Z property, Explicit substitutions, Rocq Prover, Formal verification.

1 Introduction

We present a framework for studying properties of extensions of the λ-calculus [3], which is based on a nominal approach [14]. In the nominal setting, variables are represented by atoms, structureless entities with a decidable equality. Therefore different names mean different atoms and different variables. The nominal approach is close to the usual paper and pencil notation used in λ-calculus, whose grammar of terms is given by:

$$t ::= x \mid \lambda_x.t \mid t\ t \tag{1}$$

where x represents a variable taken from an enumerable set, $\lambda_x.t$ is an abstraction, and $t\ t$ is an application. The abstraction is the only binding operator: in the expression $\lambda_x.t$, x binds in t, called the scope of the abstraction. This means that every free occurrence of x in t becomes bound in $\lambda_x.t$. A variable that is

not in the scope of an abstraction is free. A variable in a term is either bound or free, but note that a variable can occur both bound and free in the same term, as in $(\lambda_y.y)\ y$. The main rule of the λ-calculus, named β-reduction, is given by:

$$(\lambda_x.t)\ u \rightarrow_\beta \{x := u\}t \tag{2}$$

where $\{x := u\}t$ represents the result of substituting all free occurrences of variable x in t with u, renaming bound variables whenever needed to avoid capturing free variables of u. We call t the body of the metasubstitution, and u its argument. In other words, $\{x := u\}t$ is a metanotation for a capture-free substitution. For instance, the λ-term $(\lambda_x\lambda_y.x\ y)\ y$ has both bound and free occurrences of the variable y, and in order to β-reduce it, one has to substitute the free variable y for all free occurrences of the variable x in the term $(\lambda_y.x\ y)$. A straight substitution would capture the free variable y, *i.e.* the free occurrence of y before the β-reduction would become bound after the β-reduction step. A renaming of bound variables avoids such a capture: in this example, one can take an α-equivalent[1] term, say $(\lambda_z.x\ z)$, and perform the β-step correctly as $(\lambda_x\lambda_y.x\ y)\ y \rightarrow_\beta \lambda_z.y\ z$. Renaming of variables in the nominal setting is done via a name-swapping, which is formally defined at the level of variables as follows:

$$(x\ y)z := \begin{cases} y, & \text{if } z = x; \\ x, & \text{if } z = y; \\ z, & \text{otherwise.} \end{cases}$$

This notion can be extended to the level of terms in a straightforward way:

$$(x\ y)t := \begin{cases} (x\ y)z, & \text{if } t = z; \\ \lambda_{(x\ y)z}.(x\ y)t_1, & \text{if } t = \lambda_z.t_1; \\ (x\ y)t_1\ (x\ y)t_2, & \text{if } t = t_1\ t_2 \end{cases} \tag{3}$$

In the previous example, the variable capture can be avoided by applying a swap to the body of the abstraction before performing the metasubstitution: $(\lambda_x\lambda_y.x\ y)\ y \rightarrow_\beta \{x := y\}((y\ z)(\lambda_y.x\ y)) = \{x := y\}(\lambda_z.x\ z) = \lambda_z.y\ z$. The metasubstitution can be defined in different ways; in our formalization, it is defined as follows:

$$\{x := u\}t = \begin{cases} u, & \text{if } t = x; \\ t, & \text{if } t = y \text{ and } x \neq y; \\ \{x := u\}t_1\ \{x := u\}t_2, & \text{if } t = t_1\ t_2; \\ \lambda_x.t_1, & \text{if } t = \lambda_x.t_1; \\ \lambda_z.(\{x := u\}((y\ z)t_1)), & \text{if } t = \lambda_y.t_1, x \neq y \\ & \text{and} \\ & z \notin fv(t\ u) \cup \{x\}. \end{cases} \tag{4}$$

where $fv(t)$ denotes the set of free variables of t. The last case of the definition of metasubstitution is the one that avoids variable capture by applying a swap to the body of the abstraction before performing the metasubstitution. In this case, z is a fresh variable that does not occur free in t or in u, and it can be

[1] A formal definition of this notion will be given later in this section.

chosen arbitrarily. In our formalization, we will use a function called *atom_fresh* to choose such a fresh variable.

The main contributions of this work are as follows:

- formalization of the compositional Z property in the nominal setting;
- formalization of the confluence property of the λ_x-calculus with explicit substitutions via the compositional Z property.

To the best of our knowledge this is the first formalization of the compositional Z property. The rest of the paper is organized as follows: In Section 2, we present the (compositional) Z Property. In Section 3, we present the λ_x-calculus and the confluence proof via the compositional Z property. In Section 4, we conclude and discuss future work.

1.1 An Explicit Substitution Operator

In this section, we extend the grammar (1) with a new constructor called an explicit substitution operator:

$$t ::= x \mid \lambda_x.t \mid t\ t \mid [x := u]t \tag{5}$$

where $[x := u]t$ represents a term with a pending substitution that will be evaluated by specific rules of a substitution calculus. The intended meaning of the explicit substitution is that it simulates the metasubstitution. This formalization aims to be a generic framework applicable to any calculus with explicit substitutions using a named notation for variables. Therefore, we do not fix rules for how the metasubstitution is to be simulated, but it is important to keep in mind that this is not a trivial task, since one can easily lose important properties of the original λ-calculus in the process [22,15].

Calculi with explicit substitutions are formalisms that deconstruct the metasubstitution operation into finer-grained steps, thereby functioning as an intermediary between the λ-calculus and its practical implementations: they shed light on the execution models of higher-order languages. In fact, developing a calculus with explicit substitutions that is faithful to the λ-calculus, in the sense of preserving some of its desired properties, has been the main motivation behind the long list of calculi with explicit substitutions invented over the last decades [1,30,4,8,23,17,6,9,18].

The following inductive definition corresponds to the grammar (5), where the explicit substitution constructor, named *n_sub*, has a special notation. Instead of writing *n_sub t x u*, we will write $[x := u]\ t$ similarly to (5). Accordingly, *n_sexp* denotes the set of nominal λ-expressions equipped with an explicit substitution operator, which, for simplicity, we will refer to as just "terms". Note that all code snippets are clickable and linked to their source repository.

```
Inductive n_sexp : Set :=
| n_var (x:atom)
| n_abs (x:atom) (t:n_sexp)
| n_app (t1:n_sexp) (t2:n_sexp)
| n_sub (t1:n_sexp) (x:atom) (t2:n_sexp).
```

A term is *pure* if it has no explicit substitution operator. The *size* of a term and its set of free variables *fv_nom* are defined as usual.

In the λ-calculus, terms that differ only by the name of bound variables are considered equivalent. This equivalence is called α-equivalence, and it is defined by the following inference rules (cf. [31]):

$$(aeq_var)\ \frac{}{x =_\alpha x} \qquad \frac{t_1 =_\alpha t_2}{\lambda_x.t_1 =_\alpha \lambda_x.t_2}\ (aeq_abs_same)$$

$$\frac{x \neq y \qquad x \notin fv(t_2) \qquad t_1 =_\alpha (y\ x)t_2}{\lambda_x.t_1 =_\alpha \lambda_y.t_2}\ (aeq_abs_diff)$$

$$\frac{t_1 =_\alpha t_1' \qquad t_2 =_\alpha t_2'}{t_1\ t_2 =_\alpha t_1'\ t_2'}\ (aeq_app)$$

In [19], this definition was extended to explicit substitutions by adding the following rules:

$$\frac{t_1 =_\alpha t_1' \qquad t_2 =_\alpha t_2'}{[x := t_2]t_1 =_\alpha [x := t_2']t_1'}\ (aeq_sub_same)$$

$$\frac{t_2 =_\alpha t_2' \qquad x \neq y \qquad x \notin fv(t_1') \qquad t_1 =_\alpha (y\ x)t_1'}{[x := t_2]t_1 =_\alpha [y := t_2']t_1'}\ (aeq_sub_diff)$$

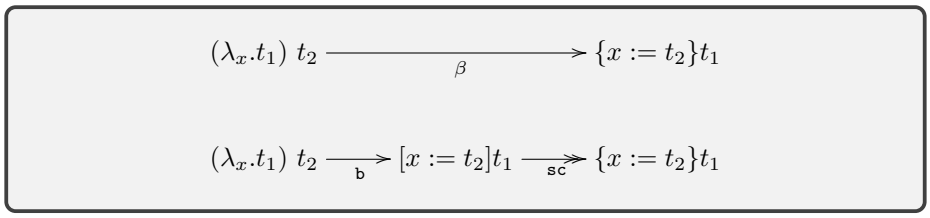
In the formalization, the resulting relation is called aeq. We prove its basic properties, reflexivity (aeq_refl), symmetry (aeq_sym) and transitivity (aeq_trans), as well as the facts that α-equivalence is stable under swapping (aeq_swap) and that α-equivalent terms have the same free variables (aeq_fv_nom). The next lemma shows how the metasubstitution propagates through an abstraction whose bound variable y is different from the variable x being replaced: up to α-equivalence, the bound variable is renamed to a fresh variable z. This renaming prevents the free variables of u from being captured by the abstraction:

```
Lemma aeq_m_subst_abs_neq: ∀ t u x y z, x ≠ y → z ∉ fv_nom u ∪ fv_nom
(n_abs y t) ∪ {{x}} → {x := u}(n_abs y t) =α n_abs z ({x := u}(swap y z t)).
```

2 The λ_x calculus with explicit substitutions

In this section, we formalize a version of the λ_x-calculus [20,21,29,5] in a nominal framework and prove its confluence via the compositional Z property [24]. This task can be seen as an application of our formalizations of the compositional Z property [10] and the extension of the substitution lemma for calculi with explicit substitutions [19].

As discussed in the introduction, calculi with explicit substitutions deconstruct the β-reduction into finer-grained steps, thereby functioning as an intermediary between the λ-calculus and its practical implementations. In the λ_x-calculus, the core idea is that β-reduction is divided into two parts, one that initiates the simulation of the β-step, and another that completes the simulation, as suggested by the following figure:



where the `b` step initiates the simulation of the β-reduction, and the `sc` steps complete the simulation of the β-reduction, where the metasubstitution $\{x := t_2\}t_1$ is extended to the explicit substitution as follows:

$$\{x := u\}t = \begin{cases} u, & \text{if } t = x; \\ t, & \text{if } t = y \text{ and } x \neq y; \\ \{x := u\}t_1\ \{x := u\}t_2, & \text{if } t = t_1\ t_2; \\ \lambda_x.t_1, & \text{if } t = \lambda_x.t_1; \\ \lambda_z.(\{x := u\}((y\ z)t_1)), & \text{if } t = \lambda_y.t_1, x \neq y \\ & \text{and} \\ & z \notin fv(t\ u) \cup \{x\}; \\ [x := \{x := u\}t_2]t_1, & \text{if } t = [x := t_2]t_1; \\ [z := \{x := u\}t_2]\{x := u\}((y\ z)t_1), & \text{if } t = [y := t_2]t_1, \\ & x \neq y \text{ and} \\ & z \notin fv(t\ u) \cup \{x\}. \end{cases} \tag{6}$$

The `b` step is inductively defined as the reduction that takes a β-redex , say $(\lambda_x\ t_1)\ t_2$ that is written as $(n_app\ (n_abs\ x\ t1)\ t2)$ in Rocq and reduces it to an explicit substitution:

ROCQ

```
Inductive betax : Rel n_sexp :=
 | step_betax : ∀ (t1 t2: n_sexp) (x: atom),
     betax (n_app (n_abs x t1) t2) ([x:=t2]t1).
```

then this reduction is done modulo α-equivalence (cf. [19]).

We write $\rightarrow_b$ for the contextual closure of the b-step modulo α-equivalence, and $\rightarrow_x$ for the contextual closure of the reduction rules of the substitution calculus that are given below, where $x \neq y$:

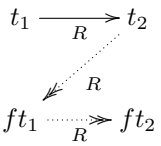
$$
\begin{array}{lll}
[y := t]y & \rightarrow_{var} & t \\
[y := t]x & \rightarrow_{gc} & x \\
[y := t_2](\lambda_y.t_1) & \rightarrow_{abs1} & \lambda_y.t_1 \\
[y := t_2](\lambda_x.t_1) & \rightarrow_{abs2} & \lambda_x.([y := t_2]t_1) \quad \text{, if } x \notin fv(t_2). \\
[y := t_2](\lambda_x.t_1) & \rightarrow_{abs3} & \lambda_z.([y := t_2](x\ z)t_1) \quad \text{, where } z \text{ is fresh and } x \in fv(t_2). \\
[y := t_3](t_1\ t_2) & \rightarrow_{app} & ([y := t_3]t_1)\ ([y := t_3]t_2)
\end{array}
\tag{7}
$$

The abstraction rule is formalized in three steps that capture the assumption that free variables cannot be captured. Therefore, in $abs1$ the term t_1 has no free occurrence of y and the substitution $[y := t_2]$ has no effect. In $abs2$, t_1 may have free occurrences of y and the substitution is propagated inside the abstraction whose bound variable x is not renamed because x has no free occurrence in t_2. Finally, in $abs3$ a renaming by a fresh variable is mandatory since x occurs free in t_2. Notice that [24] inherently handles α-equivalence, requiring only one rule for abstraction: $[y := t_2](\lambda_x.t1) \rightarrow \lambda_x.([y := t_2]t_1)$, i.e., it assumes that bound variables are renamed as needed to avoid capture of free variables.

In order to clarify the notation, we have that $\rightarrow_x = \rightarrow_{var} \cup \rightarrow_{gc} \cup \rightarrow_{abs1} \cup \rightarrow_{abs2} \cup \rightarrow_{abs3} \cup \rightarrow_{app}$ and $\rightarrow_{lx} = \rightarrow_b \cup \rightarrow_x$. In addition, the two headed arrow $\twoheadrightarrow_R$ denotes the reflexive transitive closure of the corresponding relation R.

Our goal is to prove that the λ_x-calculus is confluent, i.e., that the relation $\rightarrow_{lx}$ is confluent. To this end, we will show that the λ_x-calculus satisfies the compositional Z property, which is a sufficient condition for confluence.

The proof strategy is similar to [24], where the authors prove that the λ_x-calculus satisfies the compositional Z property, which is an extension of the Z property [27,11], a promising technique used to prove confluence of reduction systems [34,12]. In a nutshell, a function f has the Z property for a binary relation R if the following diagram holds:



For the λ_x-calculus, we take R as the reduction relation $\rightarrow_{lx}$, and f as the composition of two functions, the complete permutation P and the complete development B, recursively defined as:

```
Fixpoint P (t : n_sexp) :=
  match t with
  | n_var x ⇒ n_var x
  | n_abs x t1 ⇒ n_abs x (P t1)
  | n_app t1 t2 ⇒ n_app (P t1) (P t2)
  | n_sub t1 x t2 ⇒ {x := (P t2)}(P t1)
  end.
```

```
Fixpoint B (t : n_sexp) :=
  match t with
  | n_var x ⇒ n_var x
  | n_abs x t1 ⇒ n_abs x (B t1)
  | n_app t1 t2 ⇒
      match t1 with
      | n_abs x t3 ⇒ {x := (B t2)}(B t3)
      | _ ⇒ n_app (B t1) (B t2)
      end
  | n_sub t1 x t2 ⇒ n_sub (B t1) x (B t2)
  end.
```

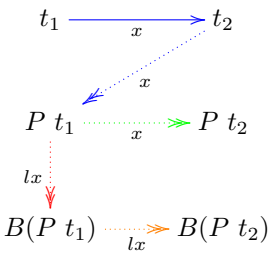
The complete permutation function P and the complete development B have several interesting properties. In what follows, we will list the most relevant ones to show how to obtain the confluence proof for the λ_x-calculus, and we will sketch the proofs of only the more interesting ones. The first point to be noticed is that $P\ t$ removes all explicit substitution of t, therefore $P\ t$ is a pure term, that is proved by induction on t:

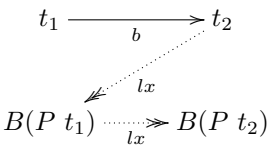
```
Lemma pure_P: ∀ t, pure (P t).
```

The B function does not introduce explicit substitutions to a pure term:

```
Lemma pure_B: ∀ t, pure t → pure (B t).
```

The Z property for the λ_x-calculus is proved via an extension called the *Compositional Z* property, as presented in [24]. It is called compositional because, under certain conditions, if the reduction relation R to be proved confluent can be decomposed into two subrelations, then it suffices to prove a Z-like property for each of them separately. In particular, for the λ_x-calculus we have $\rightarrow_{lx} = \rightarrow_b \cup \rightarrow_x$, and the proof of the compositional Z property is given by the following diagram:





(8)

The proof that the complete permutation function P is a Z function for the relation $\rightarrow_x$ (blue and green arrows) uses the fact that the explicit substitution implements the metasubstitution when the term being substituted is pure, i.e., when it does not have an explicit substitution operator. This is formalized in

the next lemma. Note that the hypothesis that *t1* is pure is needed because the λ_x-calculus does not have composition of explicit substitutions (cf. [18]), and hence the propagation of the outermost substitution $[y := t_2]$ in the term $[y := t_2]([x := t_1]t)$ must be delayed until the inner substitution $[x := t_1]$ is completely propagated.

ROCQ

```
Lemma pure_scx: ∀ t1 x t2, pure t1 → ([x := t2]t1) ⟶»x ({x := t2}t1).
```

Proof

The proof is by induction on the size of the term *t1*, and the interesting case is the abstraction case, i.e., when $t_1 = \lambda_y.t'_1$ and we need to prove that $[x := t_2](\lambda_y.t'_1) \longrightarrow\!\!\!\!\!\rightarrow_x \{x := t_2\}(\lambda_y.t'_1)$. There are two cases:

1. $(x = y)$: $[y := t_2](\lambda_y.t'_1) \longrightarrow_{abs1} \lambda_y.t'_1 = \{y := t_2\}(\lambda_y.t'_1)$.
2. $(x \neq y)$: Consider the following subcases:
   (a) Let z be a fresh variable, and suppose that $y \in fv(t_2)$. Then
   $[x := t_2](\lambda_y.t'_1) \longrightarrow_{abs3} \lambda_z.[x := t_2]((y\ z)t'_1) \longrightarrow\!\!\!\!\!\rightarrow_x^{(i.h.)}$
   $\lambda_z.\{x := t_2\}((y\ z)t'_1) \overset{(\star)}{=}_\alpha \{x := t_2\}(\lambda_y.t'_1)$.
   (b) If $y \notin fv(t_2)$ then $[x := t_2](\lambda_y.t'_1) \longrightarrow_{abs2} \lambda_y.([x := t_2]t'_1) \longrightarrow\!\!\!\!\!\rightarrow_x^{(i.h.)}$
   $\lambda_y.(\{x := t_2\}t'_1) \overset{(\star)}{=}_\alpha \{x := t_2\}(\lambda_y.t'_1)$.
   where the $(\star)$ step (in both subcases) is intuitively simple but has a tricky proof since the definition (4) of metasubstitution systematically renames variables whenever it crosses an abstraction. □

ROCQ

```
Lemma refltrans_P: ∀ t, t ⟶»x (P t).
```

Proof

The proof is by induction on t. The interesting case is when t is an explicit substitution, i.e., $t = [x := t_2]t_1$. In this case, we have to show that

$$[x := t_2]t_1 \longrightarrow\!\!\!\!\!\rightarrow_x P([x := t_2]t_1)$$

where $P([x := t_2]t_1) = \{x := P\ t_2\}(P\ t_1)$ by definition of P. This is done as follows:

$$\frac{\dfrac{}{[x := t_2]t_1 \longrightarrow\!\!\!\!\!\rightarrow_x [x := P\ t_2](P\ t_1)}(\star) \qquad \dfrac{}{[x := P\ t_2](P\ t_1) \longrightarrow\!\!\!\!\!\rightarrow_x \{x := P\ t_2\}(P\ t_1)}(\star\star)}{[x := t_2]t_1 \longrightarrow\!\!\!\!\!\rightarrow_x \{x := P\ t_2\}(P\ t_1)}\ (\mathrm{T})$$

where (T) is a transitivity step, $(\star)$ follows from the induction hypothesis for t_1 and t_2, since $\longrightarrow\!\!\!\!\!\rightarrow_x$ is compatible with explicit substitutions, and $(\star\star)$ follows from Lemma *pure_scx*, since $P\ t_1$ is pure by Lemma *pure_P*. □

Therefore, Lemma *refltrans_P* proves the blue arrows in (8). The proof that $P\ t_1 \longrightarrow\!\!\!\!\!\rightarrow_x P\ t_2$ (green arrow in (8)) is obtained by Lemma *scx_P* below. Note that the complete permutation P replaces every explicit substitution in the input term t with the corresponding metasubstitution in the output $P\,t$. Some auxiliary results are needed for *scx_P*. The important ones are:

ROCQ

```
Lemma aeq_swap_P: ∀ t x y, (P (swap x y t)) =α (swap x y (P t)).

Lemma aeq_P: ∀ t1 t2, t1 =α t2 → (P t1) =α (P t2).
```

Lemma *aeq_swap_P* is proved by induction on t, and Lemma *aeq_P*, by induction on the derivation $t1 =\alpha\ t2$. We omit their proofs to save space.

ROCQ

```
Lemma scx_P: ∀ t1 t2, (t1 →x t2) → (P t1) =α (P t2).
```

Proof

The proof is by induction on the reduction $(t1 \rightarrow_x t2)$. The non-trivial case is when $t1 \rightarrow_{abs3} t2$. In this case,

$$t1 = [y := t'_2](\lambda_x.t'_1) \rightarrow_{abs3} \lambda_z.[y := t'_2]((x\ z)t'_1) = t2,$$

where $x \neq y$ and z is a fresh variable. We proceed as follows:

$$\dfrac{\dfrac{\dfrac{}{\{y := P\ t'_2\}\lambda_x.(P\ t'_1) =_\alpha \lambda_z.\{y := P\ t'_2\}((x\ z)P\ t'_1)}\ (\diamond)}{\{y := P\ t'_2\}\lambda_x.(P\ t'_1) =_\alpha \lambda_z.\{y := P\ t'_2\}P\ ((x\ z)t'_1)}\ (aeq_swap_P)}{P\ ([y := t'_2](\lambda_x.t'_1)) =_\alpha P\ (\lambda_z.[y := t'_2]((x\ z)t'_1))}\ (\text{DEF. } P)$$

where $(\diamond)$ corresponds to an application of lemma *aeq_m_subst_abs_neq*. □

The red arrow in (8) is proved by Lemma *pure_refltrans_B* below, which is proved by induction on the size of the term t:

ROCQ

```
Lemma pure_refltrans_B: ∀ t, pure t → t ⟶»lx (B t).
```

Proof

The interesting case is when t is an application, i.e., $t = t_1\ t_2$, and we have to show that $t_1\ t_2 \longrightarrow\!\!\!\!\!\rightarrow_{lx} B(t_1\ t_2)$. If t_1 is not an abstraction, then $B(t_1\ t_2) = B(t_1)\ B(t_2)$ and the proof is by induction hypothesis. If t_1 is an abstraction, say $t_1 = \lambda_x.t'_1$, then we have to show that

$$(\lambda_x.t'_1)\ t_2 \longrightarrow\!\!\!\!\!\rightarrow_{lx} \{x := (B\ t_2)\}(B\ t'_1)$$

since $B((\lambda_x.t'_1)\ t_2) = \{x := (B\ t_2)\}(B\ t'_1)$:

$$\dfrac{\dfrac{\dfrac{\dfrac{}{\lambda_x.t'_1 \longrightarrow\!\!\!\!\rightarrow_{lx} \lambda_x.B(t'_1)}\ (\text{I.H.})}{\lambda_x.t'_1 \longrightarrow\!\!\!\!\rightarrow_{lx} B(\lambda_x.t'_1)}}{(\lambda_x.t'_1)\ t_2 \longrightarrow\!\!\!\!\rightarrow_{lx} (B(\lambda_x.t'_1))\ t_2} \qquad \dfrac{(\star)}{(B(\lambda_x.t'_1))\ t_2 \longrightarrow\!\!\!\!\rightarrow_{lx} \{x := (B\ t_2)\}(B\ t'_1)}}{(\lambda_x.t'_1)\ t_2 \longrightarrow\!\!\!\!\rightarrow_{lx} \{x := (B\ t_2)\}(B\ t'_1)}\ (\text{T})$$

where $(\star)$ is proved as follows:

$$\dfrac{(\lambda_x.B(t'_1))\ t_2 \longrightarrow\!\!\!\!\rightarrow_b [x := t_2](B\ t'_1) \qquad \dfrac{(\star\star)}{[x := t_2](B\ t'_1) \longrightarrow\!\!\!\!\rightarrow_{lx} \{x := (B\ t_2)\}(B\ t'_1)}}{\dfrac{(\lambda_x.B(t'_1))\ t_2 \longrightarrow\!\!\!\!\rightarrow_{lx} \{x := (B\ t_2)\}(B\ t'_1)}{(B(\lambda_x.t'_1))\ t_2 \longrightarrow\!\!\!\!\rightarrow_{lx} \{x := (B\ t_2)\}(B\ t'_1)}}$$

where $(\star\star)$ is proved as follows:

$$\dfrac{\dfrac{\dfrac{}{t_2 \longrightarrow\!\!\!\!\rightarrow_{lx} B\ t_2}\ (\text{I.H.})}{[x := t_2](B\ t'_1) \longrightarrow\!\!\!\!\rightarrow_{lx} [x := (B\ t_2)](B\ t'_1)} \qquad (\star\star\star)}{[x := t_2](B\ t'_1) \longrightarrow\!\!\!\!\rightarrow_{lx} \{x := (B\ t_2)\}(B\ t'_1)}$$

where the $(\star\star\star)$ corresponds to the reduction

$$[x := (B\ t_2)](B\ t'_1) \longrightarrow\!\!\!\!\rightarrow_{lx} \{x := (B\ t_2)\}(B\ t'_1)$$

that is justified by Lemma *pure_scx* since $(B\ t'_1)$ is pure by Lemma *pure_B*. □

In order to complete the proof of the left diagram in (8), we need to show that $B(P\ t_1) \longrightarrow\!\!\!\!\rightarrow_{lx} B(P\ t_2)$ (orange arrow). From Lemma *scx_P*, we have that $P\ t_1 =_\alpha P\ t_2$, and we conclude because B is stable under α-equivalence, as shown by Lemma *aeq_B* below:

```
Lemma aeq_B: ∀ t1 t2, t1 =α t2 → (B t1) =α (B t2).
```

We now proceed to the proof of the second diagram in (8). We have that $t_1 \to_b t_2$ and we want to prove that $t_2 \longrightarrow\!\!\!\!\rightarrow_{lx} B(P\ t_1)$ and $B(P\ t_1) \longrightarrow\!\!\!\!\rightarrow_{lx} B(P\ t_2)$. The first part, proved by Lemma *step_b_refltrans_lx_B_P* below, is proved by induction on the reduction $t_1 \to_b t_2$ and the tricky step is to prove that $\{x := t_2\}t_1 \longrightarrow\!\!\!\!\rightarrow_{lx} B(\{x := t_2\}t_1)$, whose solution is to use the fact that $\to_{lx}$-steps simulate $\to_\beta$-steps, that are now defined as in (2), but over *n_sexp* terms. Let us see how it works.

The simulation of one step β-reduction of pure terms is proved in the next lemma using induction on the reduction $t_1 \to_\beta t_2$:

ROCQ

```
Lemma pure_beta_lx: ∀ t1 t2, pure t1 → (t1 →β t2) → (t1 ⟶»lx t2).
```

One challenging task in this formalization was the proof of the next lemma, which states that substituting the complete developments of its components into the complete development of a term reduces (via β-reduction) to the complete development of the corresponding metasubstitution.

ROCQ

```
Lemma pure_m_subst_refltrans_beta_B: ∀ t1 t2 x, pure t1 → pure t2 →
                                ({x := B t2} B t1) ⟶»β (B ({x := t2} t1)).
```

Proof

The proof is by induction on the size of the term t_1. The interesting case is the application case. If $t_1 = t_{11}\ t_{12}$ then we need to prove that

$$\{x := B\ t_2\}(B(t_{11}\ t_{12})) \longrightarrow\!\!\!\!\!\rightarrow_\beta B(\{x := t_2\}(t_{11}\ t_{12}))$$

We proceed by case analysis on the structure of t_{11}. If t_{11} is the variable x then our goal is $(B\ t_2)\ (\{x := B\ t_2\}(Bt_{12}) \longrightarrow\!\!\!\!\!\rightarrow_\beta B\ (t_2\ (\{x := t_2\}t_{12}))$ and in turn, we proceed by case analysis on the structure of t_2. The non-trivial case, again is when $t_2 = \lambda_y.t_2'$:

$$\dfrac{\dfrac{\dfrac{\dfrac{}{\{x := B\ (\lambda_y.t_2')\}(Bt_{12}) \longrightarrow\!\!\!\!\!\rightarrow_\beta B(\{x := \lambda_y.t_2'\}t_{12})}\ \text{(i.h.)}}{\{y := (\{x := B\ (\lambda_y.t_2')\}(Bt_{12}))\}(B\ t_2') \longrightarrow\!\!\!\!\!\rightarrow_\beta \{y := (B(\{x := (\lambda_y.t_2')\}t_{12}))\}(B\ t_2')}}{(\lambda_y.B\ t_2')\ (\{x := B\ (\lambda_y.t_2')\}(Bt_{12})) \longrightarrow\!\!\!\!\!\rightarrow_\beta \{y := (B(\{x := (\lambda_y.t_2')\}t_{12}))\}(B\ t_2')}}{(B\ (\lambda_y.t_2'))\ (\{x := B\ (\lambda_y.t_2')\}(Bt_{12}) \longrightarrow\!\!\!\!\!\rightarrow_\beta B\ ((\lambda_y.t_2')\ (\{x := (\lambda_y.t_2')\}t_{12})))}$$

Another non-trivial case occurs when $t_{11} = \lambda_y.t_{11}'$, leading to the goal

$$\{x := B\ t_2\}(B((\lambda_y.t_{11}')\ t_{12})) \longrightarrow\!\!\!\!\!\rightarrow_\beta B(\{x := t_2\}((\lambda_y.t_{11}')\ t_{12}))$$

whose derivation is given by

$$\dfrac{\{x := B\ t_2\}(\{y := B\ t_{12}\}(B\ t_{11}')) \longrightarrow\!\!\!\!\!\rightarrow_\beta B(\{x := t_2\}((\lambda_y.t_{11}')\ t_{12}))}{\{x := B\ t_2\}(B((\lambda_y.t_{11}')\ t_{12})) \longrightarrow\!\!\!\!\!\rightarrow_\beta B(\{x := t_2\}((\lambda_y.t_{11}')\ t_{12}))}$$

Then we have two cases, either $x = y$ or $x \neq y$:

1. $x = y$:

$$\dfrac{\dfrac{\dfrac{\dfrac{\dfrac{}{\{x := B\ t_2\}(B\ t_{12}) \longrightarrow\!\!\!\!\!\rightarrow_\beta B\{x := t_2\}t_{12}}\ \text{(i.h.)}}{\{y := (\{x := B\ t_2\}(B\ t_{12}))\}(B\ t'_{11}) \longrightarrow\!\!\!\!\!\rightarrow_\beta \{y := B(\{x := t_2\}t_{12})\}(B\ t'_{11})}}{\{y := (\{x := B\ t_2\}(B\ t_{12}))\}(B\ t'_{11}) \longrightarrow\!\!\!\!\!\rightarrow_\beta B((\lambda_y.t'_{11})\ \{x := t_2\}t_{12})}}{\{x := B\ t_2\}(\{y := B\ t_{12}\}(B\ t'_{11})) \longrightarrow\!\!\!\!\!\rightarrow_\beta B(\{x := t_2\}((\lambda_y.t'_{11})\ t_{12}))}}{\{x := B\ t_2\}(B((\lambda_y.t'_{11})\ t_{12})) \longrightarrow\!\!\!\!\!\rightarrow_\beta B(\{x := t_2\}((\lambda_y.t'_{11})\ t_{12}))}$$

2. $x \neq y$:

$$\dfrac{\dfrac{\dfrac{\dfrac{(\star)}{\{x := B\ t_2\}(\{y := B\ t_{12}\}(B\ t'_{11})) \longrightarrow\!\!\!\!\!\rightarrow_\beta \{z := B(\{x := t_2\}t_{12})\}(B(\{x := t_2\}(z\ y)t'_{11}))}}{\{x := B\ t_2\}(\{y := B\ t_{12}\}(B\ t'_{11})) \longrightarrow\!\!\!\!\!\rightarrow_\beta B((\lambda_z.(\{x := t_2\}(z\ y)t'_{11}))\ (\{x := t_2\}t_{12}))}}{\{x := B\ t_2\}(\{y := B\ t_{12}\}(B\ t'_{11})) \longrightarrow\!\!\!\!\!\rightarrow_\beta B(\{x := t_2\}((\lambda_y.t'_{11})\ t_{12}))}}{\{x := B\ t_2\}(B((\lambda_y.t'_{11})\ t_{12})) \longrightarrow\!\!\!\!\!\rightarrow_\beta B(\{x := t_2\}((\lambda_y.t'_{11})\ t_{12}))}$$

where $(\star)$ is obtained by decomposing the reduction
$\{x := B\ t_2\}(\{y := B\ t_{12}\}(B\ t'_{11})) \longrightarrow\!\!\!\!\!\rightarrow_\beta \{z := B(\{x := t_2\}t_{12})\}(B(\{x := t_2\}(z\ y)t'_{11}))$
using $\{z := \{x := B\ t_2\}(B\ t_{12})\}(\{x := B\ t_2\}((z\ y)(B\ t'_{11})))$ as the intermediate term, and each subreduction is solved as follows: the first subreduction is an instance of the Substitution Lemma (*m_subst_lemma*) formalized in [19], and the second subreduction is obtained by applying the induction hypothesis to the two components of the application:

$$\dfrac{\dfrac{}{\{x := B\ t_2\}((z\ y)(B\ t'_{11})) \longrightarrow\!\!\!\!\!\rightarrow_\beta B(\{x := t_2\}(z\ y)t'_{11})} \quad \dfrac{}{\{x := B\ t_2\}(B\ t_{12}) \longrightarrow\!\!\!\!\!\rightarrow_\beta B(\{x := t_2\}t_{12})}}{\{z := \{x := B\ t_2\}(B\ t_{12})\}(\{x := B\ t_2\}((z\ y)(B\ t'_{11}))) \longrightarrow\!\!\!\!\!\rightarrow_\beta \{z := B(\{x := t_2\}t_{12})\}(B(\{x := t_2\}(z\ y)t'_{11}))}$$

□

Now we are ready to prove the first part of the second diagram in (8). As explained before, the challenging case is the one involving the explicit substitution:

ROCQ

```
Lemma step_b_refltrans_lx_B_P: ∀ t1 t2, t1 →b t2 → t2 ⟶»lx (B(P t1)).
```

Proof

Suppose that

$$t_1 = [x := t_{12}]t_{11} \rightarrow_b [x := t_{12}]t'_{11} = t_2, \text{ with } t_{11} \rightarrow_b t'_{11}$$

or

$$t_1 = [x := t_{12}]t_{11} \rightarrow_b [x := t'_{12}]t_{11} = t_2, \text{ with } t_{12} \rightarrow_b t'_{12}.$$

Both cases have similar proofs, therefore we consider only the first reduction that proceeds as follows:

$$\dfrac{\dfrac{\dfrac{\dfrac{}{t'_{11} \twoheadrightarrow_{lx} B(P\ t_{11})} \quad \dfrac{(\star)}{t_{12} \twoheadrightarrow_{lx} B(P\ t_{12})}}{[x := t_{12}]t'_{11} \twoheadrightarrow_{lx} [x := B(P\ t_{12})]B(P\ t_{11})}}{[x := t_{12}]t'_{11} \twoheadrightarrow_{lx} B([x := P\ t_{12}](P\ t_{11}))} \quad \dfrac{}{B([x := P\ t_{12}](P\ t_{11})) \twoheadrightarrow_{lx} B(\{x := P\ t_{12}\}(P\ t_{11}))}\ (\star\star)}{[x := t_{12}]t'_{11} \twoheadrightarrow_{lx} B(\{x := P\ t_{12}\}(P\ t_{11}))}}{[x := t_{12}]t'_{11} \twoheadrightarrow_{lx} B(P([x := t_{12}]t_{11}))}$$

where $(\star)$ is easily proved by lemmas *refltrans_P*, *pure_P* and *pure_refltrans_B*, and $(\star\star)$ is proved as follows:

$$\dfrac{\dfrac{\dfrac{}{[x := B(P\ t_{12})](B(P\ t_{11})) \twoheadrightarrow_{lx} \{x := B(P\ t_{12})\}(B(P\ t_{11}))}\ (\star\star\star)}{[x := B(P\ t_{12})](B((P\ t_{11}))) \twoheadrightarrow_{lx} B(\{x := P\ t_{12}\}(P\ t_{11}))}}{B([x := P\ t_{12}](P\ t_{11})) \twoheadrightarrow_{lx} B(\{x := P\ t_{12}\}(P\ t_{11}))}\ (\star\star\star\star)$$

where $(\star\star\star)$ is proved by lemma *pure_pix* since $\rightarrow_x \subseteq \rightarrow_{lx}$, and $(\star\star\star\star)$ is given by the reduction

$$\{x := B(P\ t_{12})\}(B((P\ t_{11}))) \twoheadrightarrow_{lx} B(\{x := P\ t_{12}\}(P\ t_{11})) \tag{9}$$

which is the tricky part of the proof. It is proved using the fact that $\rightarrow_{lx}$ steps can simulate β-steps as proved by Lemma *pure_beta_lx*. □

Finally, the last part of the second diagram in (8) is proved by the next lemma:

ROCQ

```
Lemma refltrans_lx_B_P: ∀ t1 t2, (t1 →b t2) → (B (P t1)) ⟶»lx (B (P t2)).
```

$$\dfrac{\dfrac{\dfrac{t_1 \rightarrow_b t_2}{P\ t_1 \twoheadrightarrow_\beta P\ t_2}}{B(P\ t_1) \twoheadrightarrow_\beta B(P\ t_2)}}{B(P\ t_1) \twoheadrightarrow_{lx} B(P\ t_2)}$$

□ The diagrams in (8) are formalized as the predicate *Z_comp_aeq* used in the next lemma:

ROCQ

```
Lemma lambda_x_Z_comp_aeq: Z_comp_aeq lambdax.
```

Therefore, we can conclude that the reduction $\rightarrow_{lx}$ satisfies the compositional Z-property modulo α-equivalence. In [10], we formalized that the compositional

Z property implies the Z property, which in turn implies confluence. These intermediate results are used in the next theorem to conclude that $\to_{lx}$ is confluent.

```
Theorem lambda_x_is_confluent: Confl lambdax.
Proof.
  apply Z_prop_aeq_implies_Confl.
  apply Z_comp_aeq_implies_Z_prop_aeq.
  apply lambda_x_Z_comp_aeq.
Qed.
```

3 Related Work and Conclusion

Related Work. The Z property was introduced by van Oostrom [27] and Dehornoy and van Oostrom [11] as a simple, self-contained alternative to the classical Tait-Martin-Löf technique for proving confluence [16,25,26], and it has since been used as a benchmark for confluence tools and formalizations [12,34]. Nakazawa and Fujita [24] proposed the compositional Z property, an extension tailored to calculi whose reduction relation splits naturally into an initiating step and a substitution calculus that completes it, and used it to give a pencil-and-paper confluence proof for a calculus with explicit substitutions close to the λ_x-calculus studied here. The present paper can be seen as a full mechanization, and a nominal reformulation, of that proof strategy.

Our development directly builds on two previous formalizations by (a subset of) the same authors: [10], which formalizes the Z and compositional Z properties, and [19], which extends the classical substitution lemma to calculi with explicit substitutions. Both were carried out using a locally nameless representation of binders, built on top of the `Metalib` library [35]. Here we keep the same underlying atom infrastructure but switch to a nominal representation, in which α-equivalence is defined explicitly via name-swapping [14,28] rather than being identified with syntactic equality of De Bruijn-style terms. This choice trades some of the automation that locally nameless representations provide for a syntax that is closer to the informal, textbook presentation of binders, which we found particularly convenient when stating and proving properties of the metasubstitution and of the λ_x-calculus reduction rules. Related nominal infrastructures have been developed in other proof assistants, notably Nominal Isabelle [33] and a nominal library for Agda [13]; unlike these, our formalization does not rely on a dedicated nominal package or on quotient types, but implements swapping and α-equivalence directly as inductive definitions over raw terms, in the same spirit as other machine-checked accounts of the Barendregt variable convention [7,32,2].

Conclusion. We presented a nominal formalization, in the Rocq proof assistant (version 9.1.1), of a framework for proving confluence via the (compositional) Z property, together with the supporting theory of α-equivalence, swapping, and capture-avoiding metasubstitution for nominal terms with an explicit substitution operator. As an application of this framework, we formalized the λ_x-calculus

and proved that its reduction relation $\rightarrow_{lx}$ is confluent, by exhibiting the complete permutation function P and the complete development function B as witnesses of the compositional Z property. To the best of our knowledge, this is the first fully mechanized, nominal proof of confluence for a calculus with explicit substitutions via the (compositional) Z property. The whole development is freely available (https://github.com/flaviodemoura/Zprop-nominal.git) and comprises around 4 500 lines of Rocq code, organized in 150 lemmas and theorems, which can serve as a reusable toolkit for confluence proofs of other calculi with explicit substitutions.

As future work, we intend to extend the framework with metavariables, which play an important role in the study of unification, and to apply it to other calculi with explicit substitutions from the long list surveyed in the introduction [1,30,4,8,23,17,6,9,18], including calculi that support composition of substitutions, a feature the λ_x-calculus deliberately avoids. We also plan a systematic, quantitative comparison between the nominal formalization presented here and the locally nameless formalizations [10,19] it builds upon, in terms of proof size, automation effort, and robustness to changes in the calculus under study.

References

1. Abadi, M., Cardelli, L., Curien, P.L., Lévy, J.J.: Explicit Substitutions. Journal of Functional Programming **1**(4), 375–416 (1991). https://doi.org/10.1017/S0956796800000186
2. Aydemir, B., Charguéraud, A., Pierce, B.C., Pollack, R., Weirich, S.: Engineering Formal Metatheory. ACM SIGPLAN Notices **43**(1), 3 (2008). https://doi.org/10.1145/1328897.1328443
3. Barendregt, H.P.: The Lambda Calculus: Its Syntax and Semantics. No. v. 103 in Studies in Logic and the Foundations of Mathematics, North-Holland ; Sole distributors for the U.S.A. and Canada, Elsevier Science Pub. Co, Amsterdam ; New York : New York, N.Y, rev. ed edn. (1984)
4. Benaissa, Z.E.A., Briaud, D., Lescanne, P., Rouyer-Degli, J.: $\lambda\nu$, a calculus of explicit substitutions which preserves strong normalisation. Journal of Functional Programming **6**(5), 699–722 (Sep 1996). https://doi.org/10.1017/S0956796800001945
5. Bloo, R., Rose, K.: Preservation of Strong Normalisation in Named Lambda Calculi with Explicit Substitution and Garbage Collection. In: CSN-95: COMPUTER SCIENCE IN THE NETHERLANDS. pp. 62–72 (1995)
6. Bloo, R., Geuvers, H.: Explicit Substitution: On the Edge of Strong Normalization. Theoretical Computer Science **211**(1-2), 375–395 (1999). https://doi.org/10.1016/s0304-3975(97)00183-7
7. Copello, E., Szasz, N., Tasistro, Á.: Machine-checked Proof of the Church-Rosser Theorem for the Lambda Calculus Using the Barendregt Variable Convention in Constructive Type Theory. Electronic Notes in Theoretical Computer Science **338**, 79–95 (Oct 2018). https://doi.org/10.1016/j.entcs.2018.10.006
8. Curien, P.L., Hardin, T., Lévy, J.J.: Confluence Properties of Weak and Strong Calculi of Explicit Substitutions. Journal of the ACM **43**(2), 362–397 (1996). https://doi.org/10.1145/226643.226675

9. David, R., Guillaume, B.: A lambda-calculus with explicit weakening and explicit substitution. Mathematical Structures in Computer Science **11**(1), 169–206 (2001). https://doi.org/10.1017/S0960129500003224
10. de Moura, F.L.C., Rezende, L.O.: A formalization of the (compositional) Z property. In: Fifth Workshop on Formal Mathematics for Mathematicians (2021)
11. Dehornoy, P., van Oostrom, V.: Z, proving confluence by monotonic single-step upperbound functions. Logical Models of Reasoning and Computation (LMRC-08) p. 85 (2008)
12. Felgenhauer, B., Nagele, J., van Oostrom, V., Sternagel, C.: The Z property. Archive of Formal Proofs (Jun 2016)
13. Gabbay, M.J., Melkonian, O.: Nominal techniques as an Agda library (Mar 2026). https://doi.org/10.48550/arXiv.2603.03968
14. Gabbay, M.J., Pitts, A.M.: A New Approach to Abstract Syntax with Variable Binding. Formal Aspects of Computing **13**(3-5), 341–363 (Jul 2002). https://doi.org/10.1007/s001650200016
15. Guillaume, B.: The λ_{se}-calculus does not preserve strong normalisation. Journal of Functional Programming **10**(4), 321–325 (Jul 2000). https://doi.org/10.1017/S0956796800003695
16. Honda, Y., Nakazawa, K., Fujita, K.e.: Confluence Proofs of Lambda-Mu-Calculi by Z Theorem. Studia Logica **109**(5), 917–936 (Oct 2021). https://doi.org/10.1007/s11225-020-09931-0
17. Kamareddine, F., Ríos, A.: Extending a λ-calculus with explicit substitution which preserves strong normalisation into a confluent calculus on open terms. Journal of Functional Programming **7**(4), 395–420 (Jul 1997). https://doi.org/10.1017/S0956796897002785
18. Kesner, D.: A Theory of Explicit Substitutions with Safe and Full Composition. Logical Methods in Computer Science **Volume 5, Issue 3**, 816 (Jul 2009). https://doi.org/10.2168/LMCS-5(3:1)2009
19. Lima, M.J.D., de Moura, F.L.C.: A Formalized Extension of the Substitution Lemma in Coq. EPTCS **389**, 80–95 (2023). https://doi.org/10.4204/EPTCS.389.7
20. Lins, R.: A new formula for the execution of categorical combinators. 8th Conference on Automated Deduction (CADE) **volume 230 of LNCS**, 89–98 (1986)
21. Lins, R.: Partial categorical multi-combinators and Church Rosser theorems. Tech. Rep. 7/92, Computing Laboratory, University Kent at Canterbury (May 1992)
22. Mellies, P.A.: Typed λ-calculi with explicit substitutions may not terminate. In: Goos, G., Hartmanis, J., Van Leeuwen, J., Dezani-Ciancaglini, M., Plotkin, G. (eds.) Typed Lambda Calculi and Applications, vol. 902, pp. 328–334. Springer Berlin Heidelberg, Berlin, Heidelberg (1995). https://doi.org/10.1007/BFb0014062
23. Muñoz, C.A.: Confluence and Preservation of Strong Normalisation in an Explicit Substitutions Calculus. In: Proceedings, 11th Annual IEEE Symposium on Logic in Computer Science, New Brunswick, New Jersey, USA, July 27-30, 1996. pp. 440–447 (1996). https://doi.org/10.1109/LICS.1996.561460
24. Nakazawa, K., Fujita, K.e.: Compositional Z: Confluence Proofs for Permutative Conversion. Studia Logica **104**(6), 1205–1224 (2016). https://doi.org/10.1007/s11225-016-9673-0
25. Nakazawa, K., Fujita, K.e.: Z for call-by-value. In: 6th International Workshop on Confluence (IWC 2017). pp. 57–61 (2017)

26. Nakazawa, K., Fujita, K.e., Imagawa, Y.: Z property for the shuffling calculus. Mathematical Structures in Computer Science pp. 1–13 (Jan 2023). https://doi.org/10.1017/S0960129522000408
27. van Oostrom, V.: Z - Draft: For Your Mind Only (2007)
28. Pitts, A.M.: Nominal logic, a first order theory of names and binding. Information and Computation **186**(2), 165–193 (Nov 2003). https://doi.org/10.1016/S0890-5401(03)00138-X
29. Rose, K.: Explicit cyclic substitutions. In: Rusinowitch, M., Rémy, J.L. (eds.) 3rd International Workshop on Conditional Term Rewriting Systems (CTRS). vol. 656, pp. 36–50. Springer-Verlag (1992)
30. Rose, K.H., Bloo, R., Lang, F.: On Explicit Substitution With Names. J Autom Reasoning **49**(2), 275–300 (2011). https://doi.org/10.1007/s10817-011-9222-5
31. Urban, C., Pitts, A.M., Gabbay, M.J.: Nominal Unification. Theoretical Computer Science **323**(1-3), 473–497 (2004). https://doi.org/10.1016/j.tcs.2004.06.016
32. Urban, C., Berghofer, S., Norrish, M.: Barendregt's Variable Convention in Rule Inductions. In: Pfenning, F. (ed.) Automated Deduction – CADE-21. pp. 35–50. Springer, Berlin, Heidelberg (2007). https://doi.org/10.1007/978-3-540-73595-3_4
33. Urban, C., Kaliszyk, C.: General Bindings and Alpha-Equivalence in Nominal Isabelle. Logical Methods in Computer Science **Volume 8, Issue 2** (Jun 2012). https://doi.org/10.2168/LMCS-8(2:14)2012
34. van Oostrom, V.: Z; syntax-free developments. In: Kobayashi, N. (ed.) 6th International Conference on Formal Structures for Computation and Deduction (FSCD 2021). Leibniz International Proceedings in Informatics (LIPIcs), vol. 195, pp. 24:1–24:22. Schloss Dagstuhl – Leibniz-Zentrum für Informatik, Dagstuhl, Germany (2021). https://doi.org/10.4230/LIPIcs.FSCD.2021.24
35. Weirich, S., al., et.: Coq Metalib package. https://github.com/plclub/metalib